

Guía de Integración con APIs Externas

Documento técnico de referencia para conectar aplicaciones al microservicio de autenticación **auth-ms**: autenticación cruzada por redirección (SSO), consumo de la API REST con clave de servicio y conexión TCP servicio-a-servicio. Incluye el catálogo completo de comandos TCP.

3

Vías de integración documentadas: SSO, REST y TCP

44

Comandos TCP disponibles en 7 dominios

3010

Puerto REST · **3011** Puerto TCP

Sistema

auth-ms

Stack

NestJS 11 · MongoDB

Consumidores

Educativa · WhatsApp · CRM · Helpdesk

Contenido

Referencia completa para integrar aplicaciones externas con la plataforma de autenticación.

- 01** Panorama general y puertos
 - Arquitectura y flujo de datos
 - Cuándo usar cada vía
- 04** Variables de entorno
 - Configuración de auth-ms
 - Configuración del cliente
- 07** Vía A — Autenticación cruzada (SSO)
 - Los 6 pasos del flujo
 - Código listo para copiar
 - Whitelist de orígenes
- 11** Vía B — API REST con clave de servicio
 - Headers requeridos
 - Catálogo de endpoints
 - Ejemplos curl
- 15** Vía C — Conexión TCP
 - Módulo cliente (simple y TLS)
 - Catálogo completo de comandos
 - Script de prueba
- 19** Payload del JWT y errores
 - Claims del token ligero
 - Códigos de error de login
- 22** Políticas y límites por tenant
 - Estructura de la configuración
 - Patrón de consumo y caché
- 25** Alta de una aplicación nueva
 - Checklist paso a paso
- 27** Pruebas y troubleshooting
 - Comandos de verificación
 - Tabla de síntomas y soluciones
- 30** Referencias y documentación viva

01 Panorama general y puertos

Qué es `auth-ms` y las tres formas de conectarse a él

auth-ms es el microservicio central de identidad y autorización (IAM) de BPONET. Administra usuarios, roles, permisos, módulos, empresas (tenants), sesiones y políticas por empresa. Todas las demás aplicaciones del ecosistema **consumen** sus servicios en lugar de mantener su propia lógica de autenticación.



Los dos puertos de auth-ms

Puerto	Protocolo	Uso	Credencial
3010	HTTP / HTTPS	API REST. Es la superficie pública: login, refresh, perfil, políticas, administración.	Authorization: Bearer x-service-key
3011	TCP (JSON)	Canal interno entre microservicios. Más eficiente para llamadas backend-a-backend frecuentes.	serviceKey (en el payload)

Fuente: `auth-ms/src/main.ts:43` (`globalPrefix api`, puerto 3010) y `auth-ms/src/main.ts:139-153` (TCP 3011)

Cuándo usar cada vía

- SSO redirección → Login de usuario
- REST + x-service-key → Integración simple
- TCP + serviceKey → Backends, alto volumen

Vía	Cuándo usarla	Ventajas
A • SSO Redirección	Login de personas desde el navegador	Usuario final nunca ve dos formularios. El token nunca pasa por la barra de direcciones de una página intermedia.
B • REST x-service-key	Integraciones de baja o media frecuencia, prototipos, clientes externos, cualquier lenguaje	Simple de depurar con <code>curl</code> o Postman. No requiere cliente TCP.
C • TCP serviceKey	Backends NestJS con llamadas frecuentes (validar JWT en cada request, consultar políticas)	Menor latencia, sin throttling HTTP y ya es el estándar en el ecosistema (educative-backend, api-whatsapp).

Principio de diseño del token

El JWT contiene **solo identidad** (`_id` , `name` , `email` , `company` , `tenantId` , `isSuperAdmin`). El árbol de **autorización** (modules/roles/permissions) se obtiene después con `GET /api/users/profile` . Esto mantiene el token por debajo de 1 KB y evita los errores **431** y **414** al pasarlo por la URL. Ver sección 6.

02 Variables de entorno

Qué debe tener configurado el servidor y qué debe tener tu aplicación

2.1 Configuración de auth-ms (servidor)

Validadas al arrancar con Joi; si falta una obligatoria, el servicio **no levanta** (fail-fast).

Variable	Req.	Default	Descripción
MONGO_URI	Sí	—	Cadena de conexión a MongoDB.
JWT_SECRET	Sí	—	Firma de los access tokens.
JWT_REFRESH_SECRET	Sí	—	Firma de los refresh tokens.
EMAIL_HOST EMAIL_USERNAME EMAIL_PASSWORD	Sí	—	SMTP para invitaciones, recuperación y avisos de nuevo login.
SSO_ALLOWED_ORIGINS	Sí	—	Whitelist de redirección SSO , separada por comas. Ej.: <code>crm-educative.bponet.com.co,api-whatsapp.bponet.com.co</code>
SERVICE_API_KEY	Sí	—	Secreto compartido entre servicios. Debe coincidir exactamente con el de cada consumidor.
PORT	No	3010	Puerto HTTP.
MICROSERVICE_HOST	No	127.0.0.1	Host de escucha TCP.
MICROSERVICE_PORT	No	3011	Puerto TCP.
JWT_ACCESS_EXPIRATION	No	1h	Vigencia del access token.
JWT_REFRESH_EXPIRATION	No	7d	Vigencia del refresh token.
CORS_ORIGIN	No	*	Orígenes permitidos, separados por coma.
TLS_ENABLED	No	false	Activa TLS en el canal TCP.
TLS_KEY_PATH TLS_CERT_PATH TLS_CA_PATH	No	—	Rutas a clave, certificado y CA.
TLS_MUTUAL	No	false	mTLS: exige requestCert y rejectUnauthorized.
TENANT_CONFIG_EMBED_IN_AUTH	No	true	Incluye la config del tenant en la respuesta de login/profile.
TEST_USER_DAYS	No	7	Días de prueba. 0 = sin expiración.

Fuente: `auth-ms/src/core/config/env.validation.ts:3-62`

2.2 Configuración del cliente (tu aplicación)

```
# — Conexión TCP con auth-ms —————
USER_MS_HOST=app.bponet.com.co
USER_MS_PORT=3011

# Secreto compartido con auth-ms. DEBE coincidir exactamente.
SERVICE_API_KEY=clave-secreta-compartida

# TLS / mTLS del canal TCP (dejar en false hasta que el servidor lo active)
TLS_ENABLED=false
TLS_CA_PATH=
TLS_KEY_PATH=
TLS_CERT_PATH=
```

Basado en nuevaInfra/api-whatsapp/apps/api/.env.example

El nombre de la variable no es estándar, es una convención del ecosistema

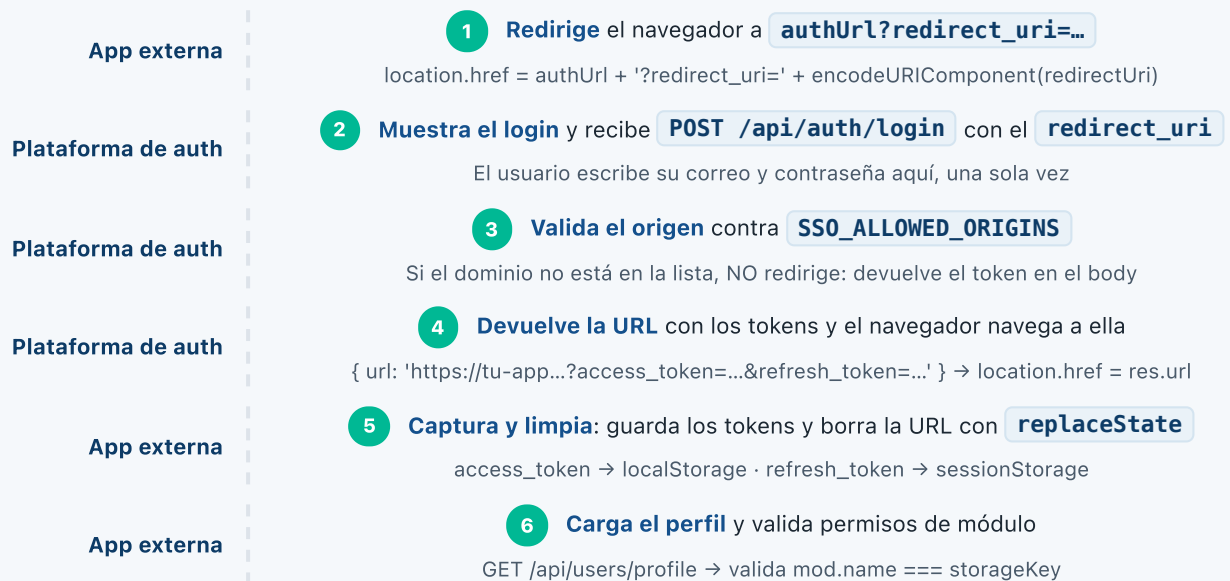
Aunque el microservicio se llame `auth-ms`, las variables del cliente se llaman `USER_MS_HOST` / `USER_MS_PORT` (heredadas de cuando el servicio se llamaba user-ms) y el cliente TCP se registra con el token `'USER_SERVICE'`. Mantén estos nombres para ser consistente con **educative-backend** y **api-whatsapp**.

03 Vía A — Autenticación cruzada por redirección (SSO)

Login centralizado: el usuario se autentica una vez y aterriza en tu aplicación

Este es el patrón que usa **crm-educative** (plataforma educativa). Tu aplicación **no recibe ni valida la contraseña**: envía al usuario al login de la plataforma de autenticación y recibe los tokens de vuelta en la URL.

SECUENCIA DEL FLUJO SSO



3.1 Configuración de la aplicación

Lo primero es declarar dos URLs en tu `environment`:

```
// environment.ts
export const ENVIRONMENT = {
  production: false,
  authUrl: 'https://auth.bponet.com.co/login', // login de la plataforma de auth
  redirectUri: 'http://localhost:4201', // origen de TU app (sin ruta)
  url: 'https://app.bponet.com.co/auth/api', // base de la API REST
  storageKey: 'crmEducative', // nombre exacto del módulo en auth-ms
};
```

Basado en `platformEducativa/crm-educative/src/environment/environment.ts`

Las dos URLs deben ser consistentes entre sí

`redirectUri` es el **origen** de tu app (sin ruta). El destino exacto se arma después añadiendo la ruta del dashboard. En producción ambos deben coincidir con el dominio registrado en `SSO_ALLOWED_ORIGINS`.

3.2 Los 6 pasos, con el código

1

Paso 1 — Lanzar la redirección

Desde un botón o un `ngOnInit` si no hay sesión, lleva al usuario al login central.

```
goToLogin() {  
  const redirectUri = ENVIRONMENT.redirectUri + '/pages/dashboard';  
  window.location.href = `${this.authUrl}?redirect_uri=${encodeURIComponent(redirectUri)}`;  
}
```

Origen: platformEducativa/crm-educative/src/app/auth/login-landing/login-landing.component.ts:229-232

2

Paso 2 — Capturar el token en el guard

Tu ruta protegida debe leer `access_token` de la URL. Es crítico **limpiar la URL** con `replaceState` para no dejar el token expuesto en el historial, en los logs del navegador y en el botón "atrás".

```
async canActivate(route, state) {  
  const accessTokenFromUrl = route.queryParamMap.get('access_token');  
  const refreshTokenFromUrl = route.queryParamMap.get('refresh_token');  
  
  if (accessTokenFromUrl) {  
    const isValid = await this.processAuthData.proccesAuthData(  
      accessTokenFromUrl, refreshTokenFromUrl ?? undefined  
    );  
    if (isValid) {  
      // Limpia el token de la barra de direcciones  
      const urlWithoutToken = state.url.split('?')[0];  
      this.location.replaceState(urlWithoutToken);  
      return true;  
    }  
    return this.router.parseUrl('/exception/403');  
  }  
  
  const token = localStorage.getItem('access_token');  
  if (!token || !this.isTokenValid(token)) { this.redirectToLogin(); return false; }  
  
  // Ya está logueado: evita llamadas extra si el módulo está cargado  
  if (localStorage.getItem(ENVIRONMENT.storageKey)) return true;  
  
  // Token válido pero sin perfil: cárgalo y valida acceso  
  const loaded = await this.processAuthData.loadProfile();  
  return loaded ? true : this.router.parseUrl('/exception/403');  
}
```

Origen: platformEducativa/crm-educative/src/app/auth/auth-guard.ts:24-69

Paso 3 — Guardar la identidad

Decodifica el token (solo lectura, para pintar la UI) y guarda los claims. El **refresh token** va a **sessionStorage**: al cerrar la pestaña se descarta, lo que obliga a un login nuevo tras un tiempo de inactividad.

Clave	Almacén	Contenido
access_token	localStorage	JWT de acceso
refreshToken	sessionStorage	JWT de refresco (se descarta al cerrar la pestaña)
exp · iat	localStorage	Vencimiento y emisión, para validar sin llamar al servidor
company · tenantId	localStorage	Contexto del tenant (clave del consumo de políticas)
_id · email · userName	localStorage	Identidad para la cabecera y la UI
isSuperAdmin · isActive	localStorage	Banderas para guards de UI
roles	localStorage	Roles del usuario (tras cargar el perfil)
<storageKey>	localStorage	Árbol de rutas del módulo. Su presencia evita pedir el perfil en cada navegación.

```

async proceseAuthData(token: string, refreshToken?: string): Promise<boolean> {
  try {
    const decoded = jwtDecode<JwtPayload>(token);

    if (refreshToken) sessionStorage.setItem('refreshToken', refreshToken);

    localStorage.setItem('access_token', token);
    localStorage.setItem('exp', String(decoded.exp));
    localStorage.setItem('company', String(decoded.company));
    localStorage.setItem('tenantId', String(decoded.tenantId));
    localStorage.setItem('_id', String(decoded._id));
    localStorage.setItem('email', decoded.email);
    // ... resto de claims

    return await this.loadProfile(); // pide /users/profile y valida módulo
  } catch (error) { return false; }
}

```

Origen: platformEducativa/crm-educative/src/app/auth/service/process-auth-data.ts:18-45

Paso 4 — Cargar el perfil y validar el módulo

El token no trae permisos. Se piden a `GET /api/users/profile` y se comprueba que el usuario tenga un módulo cuyo `name` coincida exactamente con tu `storageKey`. Si no lo tiene, se muestra el mensaje de acceso denegado.

```
async loadProfile(): Promise<boolean> {
  const profile = await firstValueFrom(this.authService.getProfile());
  const data = profile?.data;

  localStorage.setItem('roles', JSON.stringify(data?.roles || []));
  const modules = data?.modules || [];

  // El nombre del módulo debe coincidir con storageKey
  const validatedModuleExists = modules.some(
    mod => mod.name === ENVIRONMENT.storageKey
  );

  if (!validatedModuleExists) {
    this.confirmService.showMessage('error',
      `No tienes permisos para acceder a esta módulo`,
      `Contacta al administrador del sistema`);
    return false;
  }

  this.saveModuleToLocalStorage(modules);
  return true;
}
```

Origen: platformEducativa/crm-educative/src/app/auth/service/process-auth-data.ts:47-81

Paso 5 — Interceptor: token automático y refresco

Un interceptor añade el **Authorization: Bearer** a cada petición. Al recibir un **401** refresca el token y **encola** las peticiones que llegan simultáneamente, para no disparar N refrescos en paralelo.

```
const token = localStorage.getItem('access_token');

let authReq = req;
if (token && !req.url.includes('/auth/refresh')) {
  authReq = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
}

return next(authReq).pipe(
  catchError((error) => {
    // Rate limit del servidor
    if (error.status === 429) { /* avisar al usuario */ }

    if (error.status === 401 && !req.url.includes('/auth/refresh')) {
      const refreshToken = sessionStorage.getItem('refreshToken');
      if (!refreshToken) { clearSessionAndRedirect(router); return throwError(() => error); }

      if (!isRefreshing) {
        isRefreshing = true;
        refreshTokenSubject.next(null);

        return authService.refreshToken(refreshToken).pipe(
          switchMap((res) => {
            const newToken = res.access_token || res.access_token || '';
            const newRefreshToken = res.refresh_token || res.refresh_token || refreshToken;

            return from(processAuthData.procesAuthData(newToken, newRefreshToken)).pipe(
              switchMap((ok) => {
                isRefreshing = false;
                refreshTokenSubject.next(newToken);
                // Reintenta la petición original con el token nuevo
                return from(next(req.clone({
                  setHeaders: { Authorization: `Bearer ${newToken}` } })));
              }
            );
          }
        );
      } else {
        // Hay un refresco en curso: esperar y reusar su resultado
        return refreshTokenSubject.pipe(
          filter((t) => t !== null), take(1),
          switchMap((newToken) => next(req.clone({
            setHeaders: { Authorization: `Bearer ${newToken}` } })));
        }
      }
      return throwError(() => error);
    }
  })
);

function clearSessionAndRedirect(router) {
  localStorage.clear();
  sessionStorage.clear();
  setTimeout(() => { window.location.href = '/'; }, 500);
}
```

Origen: platformEducativa/crm-educative/src/app/shared/interceptors/auth.interceptor.ts:31-137

6

Paso 6 — Cierre de sesión

Para revocar la sesión en el servidor (no basta con borrar el token local) llama al logout y luego limpia el almacenamiento.

```
logout() {
  const refreshToken = sessionStorage.getItem('refreshToken');
  // POST /api/auth/logout { refreshToken } → revoca la sesión en auth-ms
  return this.http.post(`${ENVIRONMENT.url}/auth/logout`, { refreshToken });
}
```

Endpoint: auth-ms/src/auth/auth.controller.ts:134-144

3.3 La whitelist de orígenes (SSO_ALLOWED_ORIGINS)

Por seguridad, auth-ms **solo redirige** si el dominio del `redirect_uri` está registrado. La comparación acepta el dominio exacto y sus subdominios:

```
const url = new URL(redirectUri);
const isAllowed = this.whitelist.some(domain =>
  url.hostname === domain || url.hostname.endsWith('.') + domain
);

if (isAllowed) {
  url.searchParams.append('access_token', result.meta.accessToken);
  url.searchParams.append('refresh_token', result.meta.refreshToken);
  return { url: url.toString(), message: 'Login successful' };
}
```

Origen: auth-ms/src/auth/auth.controller.ts:109-131

Si tu dominio no está en la lista, el flujo "se rompe" silenciosamente

auth-ms **no lanza error**: simplemente no redirige y devuelve el token en el body de la respuesta. El frontend de la plataforma de autenticación solo hace `location.href = res.url` si `res.url` existe; si no, se queda en su propio dashboard y el usuario **nunca llega a tu aplicación**. Registra tu dominio antes de probar.

3.4 Ejemplo completo con curl (login directo, sin redirección)

```
# Desde tu backend, sin navegador: pide los tokens directamente
curl -X POST "https://app.bponet.com.co/auth/api/auth/login" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "usuario@empresa.com",
    "password": "clave-segura",
    "meta": { "browser": "API", "isbrowser": false, "user_agent": "mi-backend/1.0" }
  }'
```

Respuesta `200`:

```
{
  "message": "Login successful",
  "statusCode": 200,
  "status": "Success",
  "meta": {
    "token": "jwt...",           // alias de compatibilidad
    "accessToken": "jwt...",
    "refreshToken": "jwt...",
    "mustChangePassword": false,
    "tenantConfig": { /* ... */ }
  }
}
```

Origen: auth-ms/src/auth/auth.service.ts:224-233

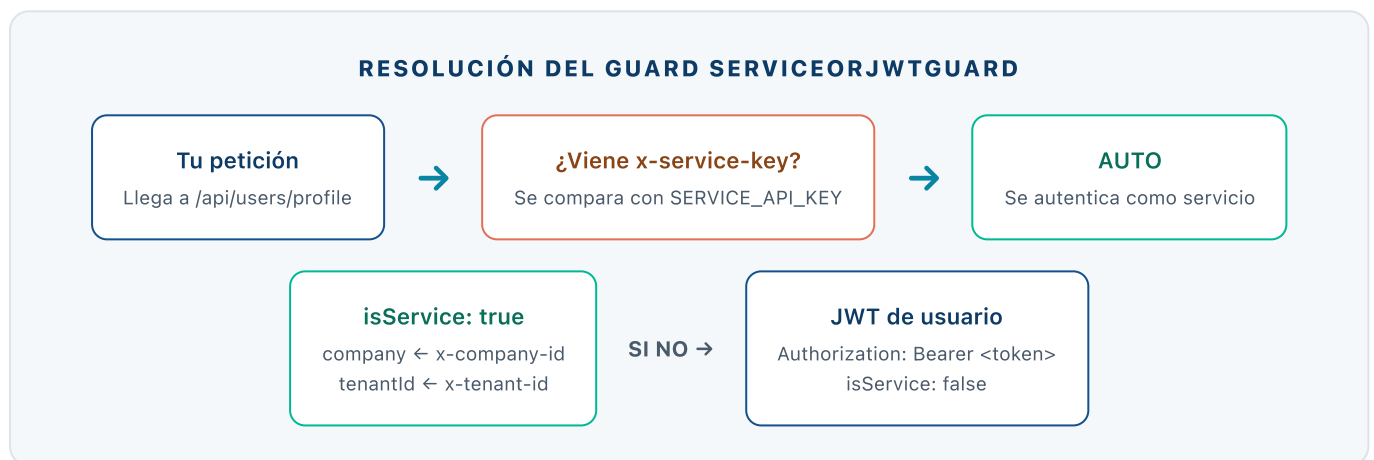
04 Vía B — API REST con clave de servicio

Integración backend-a-backend usando el secreto compartido, sin depender del JWT de un usuario

Cuando tu backend necesita consultar datos de auth-ms, puede presentar el **secreto compartido** en lugar de actuar en nombre de un usuario. Es el mecanismo que usa **api-whatsapp** para las consultas administrativas.

4.1 Autenticación: el guard dual

Las rutas marcadas con `ServiceOrJwtGuard` aceptan **dos** formas de autenticarse. El guard decide automáticamente según lo que reciba:



```
// Si el servicio presenta la clave compartida, se autentica como servicio
if (providedKey) {
  if (!this.verifyKey(providedKey)) throw new UnauthorizedException('Invalid service credentials');
  request.user = {
    isService: true, isAdmin: false, isSuperAdmin: false, isActive: true,
    company: request.headers?.['x-company-id'] || null,
    tenantId: request.headers?.['x-tenant-id'] || request.headers?.['x-company-id'] || null,
  };
  return true;
}

// Sin clave: flujo normal de frontend con JWT de usuario final
const jwtGuard = new (AuthGuard('jwt'))();
return await lastValueFrom(jwtGuard.canActivate(context));
```

Origen: auth-ms/src/core/guards/service-or-jwt.guard.ts:16-49

La comparación es en tiempo constante

La clave se valida con `crypto.timingSafeEqual`, tras comprobar que ambos buffers tienen la misma longitud. Esto evita que un atacante pueda deducir la clave midiendo cuánto tarda la respuesta. Un fallo produce **401 Unauthorized · Invalid service credentials**.

4.2 Headers requeridos

Header	Obligatorio	Uso
x-service-key	Condicional	Valor de <code>SERVICE_API_KEY</code> . Habilita el acceso como servicio en las rutas habilitadas. Si se omite, la ruta exige JWT.
x-company-id	Sí (findByTenant)	Identifica la empresa. Se usa como <code>company</code> y, si no se manda <code>x-tenant-id</code> , también como <code>tenantId</code> .
x-tenant-id	No	Tiene prioridad sobre <code>x-company-id</code> para el <code>tenantId</code> .
x-user-id	No	Sujeto sobre el que se consulta el perfil. Alternativa al query param <code>?userId=</code> .
Authorization	Condicional	<code>Bearer <access_token></code> . Alternativa a <code>x-service-key</code> .

Origen: auth-ms/src/core/guards/service-or-jwt.guard.ts:20-40 y auth-ms/src/users/users.controller.ts:189-190, 152-153

4.3 Catálogo de endpoints REST

Endpoint	Auth	Descripción
POST /api/auth/login	Pública	Inicia sesión. Acepta <code>?redirectUri=</code> para la redirección SSO.
POST /api/auth/refresh	Pública	<code>{ refreshToken }</code> → nuevo access token y nuevo <code>refreshToken</code> (rotación). Reutilizar un refresh ya rotado revoca la sesión.
POST /api/auth/logout	Pública	Revoca la sesión. Acepta <code>{ refreshToken }</code> .
POST /api/auth/change-password	JWT	Cambia la contraseña. El <code>_id</code> se toma del token, nunca del body.
POST /api/auth/recovery-password	Pública	Envía un enlace de un solo uso por correo. Respuesta genérica (no revela si el correo existe).
POST /api/auth/set-password-token	Pública	Establece la contraseña con el token de invitación/recuperación (un solo uso).
POST /api/auth/verify-email	Pública	Verifica el correo con un token de un solo uso (<code>{ token }</code>).
POST /api/auth/resent-verification	JWT	Reenvía el correo de verificación (token de un solo uso).
GET /api/auth/validate-user	JWT	Valida el token y devuelve el usuario.
GET /api/users/profile	x-service-key o JWT	Endpoint clave de integración. Identidad + modules + roles + permissions. Requiere <code>x-user-id</code> o <code>?userId=</code> si se usa la clave de servicio.
GET /api/users/profile/modules	x-service-key o JWT	Solo el árbol de módulos y rutas.
GET /api/users/profile/roles	x-service-key o JWT	Solo roles y sus permisos.
GET /api/users/profile/permissions	x-service-key o JWT	Solo la lista de permisos.
GET /api/users/findByTenant	x-service-key o JWT	Usuarios activos del tenant. Requiere <code>x-company-id</code> .
GET /api/tenants/config	x-service-key o JWT	Configuración/políticas del tenant.
POST /api/tenants/usage/report	x-service-key o JWT	Reporta consumo (deltas, idempotente por <code>reportId</code>).

Fuente: auth-ms/src/users/users.controller.ts, auth-ms/src/auth/auth.controller.ts y auth-ms/src/tenant-config/tenant-config.controller.ts

4.4 Ejemplos curl

```
# — Perfil de un usuario, autenticando como servicio —
curl "https://app.bponet.com.co/auth/api/users/profile?userId=6a15aa34beff67e3e3463c52" \
  -H "x-service-key: $SERVICE_API_KEY"

# — Perfil indicando el sujeto en el header —
curl "https://app.bponet.com.co/auth/api/users/profile" \
  -H "x-service-key: $SERVICE_API_KEY" \
  -H "x-user-id: 6a15aa34beff67e3e3463c52"

# — Usuarios de una empresa (findByTenant exige company) —
curl "https://app.bponet.com.co/auth/api/users/findByTenant" \
  -H "x-service-key: $SERVICE_API_KEY" \
  -H "x-company-id: BPONET" \
  -H "x-tenant-id: 0000000"

# — Configuración del tenant —
curl "https://app.bponet.com.co/auth/api/tenants/config?tenantId=0000000" \
  -H "x-service-key: $SERVICE_API_KEY"

# — Con JWT de usuario (flugo normal del frontend) —
curl "https://app.bponet.com.co/auth/api/users/profile" \
  -H "Authorization: Bearer $ACCESS_TOKEN"
```

Limitación conocida de CORS: x-service-key no está en la lista de headers permitidos

En main.ts la configuración de CORS declara allowedHeaders: ['Content-Type', 'Authorization', 'x-idempotency-key']. Eso significa que una llamada **desde el navegador** (fetch/XHR) con `x-service-key` será **rechazada en el preflight**. No afecta a las llamadas desde backend ni al canal TCP. Si necesitas x-service-key desde el frontend, hay que añadirlo a allowedHeaders en auth-ms.

Origen: auth-ms/src/main.ts:54-59

05 Vía C — Conexión TCP

Canal interno optimizado para llamadas entre microservicios

El canal TCP usa JSON sobre un socket persistente, con comunicación request/response por `cmd`. Es el canal que ya usan **educative-backend** y **api-whatsapp** para validar tokens en cada petición y consultar políticas.

5.1 Módulo cliente TCP

Versión básica (sin TLS), tal como la usa `educative-backend`:

```
import { Global, Module } from '@nestjs/common';
import { ClientsModule, Transport } from '@nestjs/microservices';
import { ConfigService } from '@nestjs/config';

@Global()
@Module({
  imports: [
    ClientsModule.registerAsync([
      {
        name: 'USER_SERVICE',
        useFactory: (configService: ConfigService) => ({
          transport: Transport.TCP,
          options: {
            host: configService.get<string>('USER_MS_HOST') || '127.0.0.1',
            port: configService.get<number>('USER_MS_PORT') || 3011,
          },
        }),
        inject: [ConfigService],
      },
    ]),
  ],
  exports: [ClientsModule],
})
export class TcpClientModule {}
```

Origen: platformEducativa/educative-backend/src/core/tcp/tcp.module.ts

Versión con soporte TLS/mTLS, tal como la usa `api-whatsapp`:

```

import { readFileSync } from 'fs';

function buildTlsOptions(configService: ConfigService) {
  const tlsEnabled = configService.get<string>('TLS_ENABLED') === 'true';
  if (!tlsEnabled) return undefined;

  const keyPath = configService.get<string>('TLS_KEY_PATH');
  const certPath = configService.get<string>('TLS_CERT_PATH');
  const caPath = configService.get<string>('TLS_CA_PATH');

  const tlsOptions: Record<string, Buffer> = {};
  if (caPath) tlsOptions.ca = readFileSync(caPath);
  // mTLS: el cliente presenta su propio certificado
  if (keyPath && certPath) {
    tlsOptions.key = readFileSync(keyPath);
    tlsOptions.cert = readFileSync(certPath);
  }
  return tlsOptions;
}

@Global()
@Module({
  imports: [
    ClientsModule.registerAsync([
      {
        name: 'USER_SERVICE',
        useFactory: (configService: ConfigService) => {
          const tlsOptions = buildTlsOptions(configService);
          return {
            transport: Transport.TCP,
            options: {
              host: configService.get<string>('USER_MS_HOST') || '127.0.0.1',
              port: Number(configService.get<string | number>('USER_MS_PORT') || 3011),
              ...(tlsOptions ? { tlsOptions } : {}),
            },
          };
        },
        inject: [ConfigService],
      },
    ]),
  ],
  exports: [ClientsModule],
})
export class TcpClientModule {}

```

Origen: nuevaInfra/api-whatsapp/apps/api/src/core/tcp/tcp.module.ts

Registra el módulo una sola vez en la raíz de tu app

Al estar decorated con `@Global()` y exportar `ClientsModule`, el token `'USER_SERVICE'` queda disponible en toda la aplicación. Solo necesitas importarlo una vez en tu `AppModule`.

5.2 Configuración TLS en el servidor

```
const tlsEnabled = configService.get<string>('TLS_ENABLED', 'false') === 'true';
const tlsMutual = configService.get<string>('TLS_MUTUAL', 'false') === 'true';

let tlsOptions: any;
if (tlsEnabled) {
  const keyPath = configService.get<string>('TLS_KEY_PATH');
  const certPath = configService.get<string>('TLS_CERT_PATH');
  const caPath = configService.get<string>('TLS_CA_PATH');

  if (!keyPath || !certPath) {
    throw new Error('TLS_ENABLED=true requiere TLS_KEY_PATH y TLS_CERT_PATH configurados');
  }

  tlsOptions = {
    key: fsExtra.readFileSync(keyPath),
    cert: fsExtra.readFileSync(certPath),
    ...(caPath ? { ca: fsExtra.readFileSync(caPath) } : {}),
    requestCert: tlsMutual,
    rejectUnauthorized: tlsMutual,
  };
}
```

Origen: auth-ms/src/main.ts:115-137

TLS simple

`TLS_ENABLED=true` + key y cert. Cifra el canal, pero cualquier servicio con la clave puede conectarse.

mTLS

Añade `TLS_MUTUAL=true` y el `TLS_CA_PATH`. El servidor exige certificado de cliente.

5.3 La regla de oro: serviceKey siempre

Todos los comandos TCP exigen serviceKey en el payload

Un `@MessagePattern` sin la clave válida responde `401 Invalid service credentials`. No hay excepción: es un guard global aplicado a todo handler RPC. La comparación es en tiempo constante.

```
// Canal TCP: TODOS los handlers @MessagePattern exigen serviceKey en el payload
if (context.getType() === 'rpc') {
  const data = context.switchToRpc().getData();
  return this.verifyKey(data?.serviceKey);
}

// Canal HTTP: exige la clave solo en rutas marcadas con @ServiceRoute()
const isServiceRoute = this.reflector.getAllAndOverride<boolean>(
  SERVICE_ROUTE_KEY, [context.getHandler(), context.getClass()]);

if (!isServiceRoute) return true;
```

Origen: auth-ms/src/core/guards/service-auth.guard.ts:19-39

5.4 Cómo se hacen las llamadas

Ejemplo A — educative-backend

```
userResponse = await firstValueFrom(  
  this.userClient.send(  
    { cmd: 'getUserProfile' },  
    {  
      serviceKey:  
        this.configService.get('SERVICE_API_KEY'),  
      _id: userId,  
    },  
  ),  
);
```

Origen: platformEducativa/educative-backend/src/core/services/auth.service.ts:62-69

Ejemplo B — api-whatsapp

```
const serviceKey = this.configService.get<string>  
( 'SERVICE_API_KEY' );  
if (!serviceKey) {  
  throw new UnauthorizedException('SERVICE_API_KEY  
no configurada');  
}  
  
send<JwtPayload>(  
  { cmd: 'findUserById' },  
  { serviceKey, id: _id },  
);
```

Origen: nuevaInfra/api-whatsapp/apps/api/src/core/strategies/jwt.strategy.ts:29-34

5.5 Catálogo completo de comandos TCP

Los 45 comandos disponibles, agrupados por dominio. **Todos requieren** `serviceKey`.

Revocación de sesiones: `validateUser` y `validateSession` validan la sesión contra auth-ms. Si la sesión fue revocada (o el token no tiene `sid`), `validateUser` lanza `401` y `validateSession` devuelve `{ active: false }`. Las apps externas deben tratar un `401` como cierre de sesión. No caches la validación de autenticación (o usa un TTL muy corto).

Dominio auth — auth-ms/src/auth/auth.controller.ts:343-371

cmd	Payload	Retorna
login	{ serviceKey, email, password, meta?, ip? }	Resultado del login (accessToken, refreshToken, mustChangePassword, tenantConfig)
validateUser	{ serviceKey, token }	{ user, meta: { totalData, id, valid } }. Rechaza (401) si la sesión fue revocada o el token no tiene sid.
validateSession	{ serviceKey, token }	{ active, valid, userId? } (no lanza). Útil para chequeos booleanos.
refresh	{ serviceKey, refreshToken }	Nuevo access token + payload de identidad
changePassword	{ serviceKey, id?, currentPassword, newPassword }	Confirmación del cambio

Dominio users — auth-ms/src/users/users.controller.ts:710-791

cmd	Payload	Retorna
createExternalUser	{ serviceKey, ...CreateUserDto, tenantId?, userId? }	Usuario creado (respeta límites del tenant)
createUser	{ serviceKey, ...CreateUserDto }	Objeto de creación del usuario
findAllUsers	{ serviceKey, user? }	Listado de usuarios del contexto
findUsersByTenant	{ serviceKey, user?, onlyAgents? }	Usuarios activos del tenant
findUsersByPagination	{ serviceKey, user?, page?, limit? }	Listado paginado
findUserById	{ serviceKey, id }	Un usuario
getUserProfile	{ serviceKey, _id }	Perfil con módulos, roles y permisos
findUsersByDate	{ serviceKey, user?, startDate, endDate }	Usuarios creados en el rango
updateUser	{ serviceKey, id, updateUserDto }	Usuario actualizado
removeUser	{ serviceKey, id }	Eliminación lógica
softRemoveUser	{ serviceKey, id }	Eliminación lógica explícita
hardRemoveUser	{ serviceKey, id }	Eliminación física

Nota: findUserById, removeUser, softRemoveUser y hardRemoveUser aceptan tanto { id } como el id como valor directo (payload?.id ?? payload).

Dominio roles — auth-ms/src/roles/roles.controller.ts:210-236

cmd	Payload	Retorna
createRole	{ serviceKey, ...CreateRoleDto }	Rol creado
findAllRoles	{ serviceKey }	Todos los roles (ignora el resto del payload)
findOneRole	{ serviceKey, id }	Un rol
updateRole	{ serviceKey, id, updateRoleDto }	Rol actualizado
removeRole	{ serviceKey, id }	Rol eliminado

Dominio permissions — auth-ms/src/permissions/permissions.controller.ts:222-256

cmd	Payload	Retorna
createPermission	{ serviceKey, ...CreatePermissionDto }	Permiso creado
findAllPermissions	{ serviceKey }	Todos los permisos
findOnePermission	{ serviceKey, id }	Un permiso
updatePermission	{ serviceKey, id, updatePermissionDto }	Permiso actualizado
removePermission	{ serviceKey, id }	Permiso eliminado

Dominio modules — auth-ms/src/modules/modules.controller.ts:228-256

cmd	Payload	Retorna
createModule	{ serviceKey, ...CreateModuleDto }	Módulo creado
findAllModules	{ serviceKey }	Todos los módulos
findOneModule	{ serviceKey, id }	Un módulo
updateModule	{ serviceKey, id, updateModuleDto }	Módulo actualizado
removeModule	{ serviceKey, id }	Módulo eliminado

Dominio companies — auth-ms/src/companies/companies.controller.ts:299-344

cmd	Payload	Retorna
createCompany	{ serviceKey, ...CreateCompanyDto }	Empresa creada
findAllCompanies	{ serviceKey }	Todas las empresas
findOneCompany	{ serviceKey, id }	Una empresa
updateCompany	{ serviceKey, id, updateCompanyDto }	Empresa actualizada
removeCompany	{ serviceKey, id }	Empresa eliminada
blockCompany	{ serviceKey, id, reason?, until? }	Bloquea la empresa. Sin until = indefinido
unblockCompany	{ serviceKey, id }	Desbloquea la empresa

cmd	Payload	Retorna
getTenantConfig	{ serviceKey, tenantId?, company? }	Config resuelta (defaults + valores del tenant)
getTenantPolicyCatalog	{ serviceKey }	Catálogo de definiciones de políticas
reportTenantUsage	{ serviceKey, tenantId, period, metrics, reportId }	Consumo acumulado (idempotente por reportId)
getTenantUsage	{ serviceKey, tenantId, period? }	Consumo del período
setTenantConfig	{ serviceKey, tenantId, company?, isActive?, values? }	Config actualizada (SuperAdmin)
upsertPolicyDefinition	{ serviceKey, definition }	Definición creada o editada (SuperAdmin)

5.6 Script de prueba en Node puro (sin NestJS)

Para verificar la conectividad y la clave antes de integrar en tu backend:

```
// test-tcp.js → node test-tcp.js
const net = require('net');

const payload = JSON.stringify({
  pattern: { cmd: 'getTenantPolicyCatalog' },
  data: { serviceKey: process.env.SERVICE_API_KEY },
});

const client = net.connect(
  { host: process.env.USER_MS_HOST || '127.0.0.1', port: Number(process.env.USER_MS_PORT || 3011) },
  () => { console.log('conectado, enviando...'); client.write(payload + '\n'); },
);

client.on('data', (d) => console.log('respuesta:', d.toString()));
client.on('error', (e) => console.error('error:', e.message));
client.on('close', () => client.end());
```

Si la respuesta es Invalid service credentials

El socket **conectó bien** (la red y el puerto 3011 están bien). El problema es el valor de `SERVICE_API_KEY`: no coincide con el del servidor, o no llegó en el payload.

06 Payload del JWT y contrato de errores

Qué contiene el token y por qué el login puede fallar

6.1 Claims del access token

El token contiene **únicamente identidad**. No incluye módulos, roles ni permisos:

```
export function buildIdentityPayload(user: any, sid?: string): IdentityPayload {
  return {
    _id: user._id,
    name: user.name,
    lastName: user.lastName,
    email: user.email,
    username: user.username,
    isActive: user.isActive,
    company: user.company,
    tenantId: user.tenantId,
    isSuperAdmin: user.isSuperAdmin,
    isTrial: user.isTrial === true,
    trialStartedAt: user.trialStartedAt ?? null,
    trialEndsAt: user.trialEndsAt ?? null,
    ...(sid ? { sid } : {}),
  };
}
```

Origen: auth-ms/src/auth/helpers/identity-payload.helper.ts:22-38

Por qué el token es pequeño a propósito

Históricamente el JWT incluía el árbol completo de permisos, módulos, rutas e iconos (frecuentemente > 16 KB). Al pasarlo por la URL en el flujo SSO eso provocaba **431 Request Header Fields Too Large** y **414 URI Too Long** en Node, Nginx y AWS. La separación identidad/autorización lo elimina y reduce el ancho de banda de cada petición.

6.2 Contrato de errores del login

El login puede responder **403** con **varios** motivos simultáneos, en precedencia **empresa → usuario → prueba → inactivo**:

```
{
  "message": "Tu empresa está bloqueada.",
  "code": "COMPANY_BLOCKED",
  "errors": [
    {
      "code": "COMPANY_BLOCKED",
      "message": "...",
      "details": { "blockType": "temporary", "until": "..." }
    }
  ],
  "status": "Error",
  "data": null,
  "meta": { "timestamp": "...", "path": "/api/auth/login" }
}
```

Código	Significado
COMPANY_BLOCKED	La empresa está bloqueada. Ningún usuario del tenant puede iniciar sesión.
USER_BLOCKED_TEMPORARY	Usuario bloqueado hasta una fecha (por intentos fallidos o acción de un admin).
USER_BLOCKED_INDEFINITE	Bloqueo permanente, hasta que un administrador lo levante.
TRIAL_EXPIRED	El período de prueba del usuario venció. No se muta el usuario; solo se bloquea el login.
USER_INACTIVE	El usuario existe pero está desactivado.

Fuente: CONSUMIR_TENANT_CONFIG.md §"Contrato de error de login"

6.3 Primer acceso y cambio de contraseña

Si el login responde `mustChangePassword: true`, el usuario debe pasar por `/change-password` antes de usar la aplicación. El backend bloquea los endpoints sensibles con un interceptor global mientras la bandera siga activa.

Flujo	mustChangePassword	isNewUser
Crear (invitación o contraseña temporal)	true	true
Establecer por token de invitación	false	false
Recuperación / reset de admin	true	true
Cambio de contraseña completado	false	false

`mustChangePassword` es la bandera canónica. `isNewUser` se mantiene por compatibilidad.

07 Políticas y límites por tenant

auth-ms define la política; cada aplicación la aplica en su punto de uso



7.1 Estructura de la configuración

```
{
  "tenantId": "0000000",
  "company": "BPONET",
  "version": 4,
  "updatedAt": "2026-09-24T...",

  "general": {
    "timezone": "America/Bogota", // IANA recomendado
    "locale": "es-CO"
  },

  "channels": {
    "sms": { "enabled": true, "monthlyLimit": 3000 },
    "audio": { "enabled": true, "monthlyLimit": 1000 }
  },

  "messages": {
    "bolsa": {
      "utilidad": 1000,
      "marketingComercial": 1000,
      "autenticacion": 1000,
      "servicio": 1000
    }
  },

  "features": {
    "crmInstitucional": true,
    "pbx": false
  },

  "limits": {
    "maxUsers": 0, // 0 = ilimitado
    "roles": { "AGE": 50 },
    "maxChatbotFlows": 20,
    "trialDays": 7
  },

  "security": {
    "maxFailedAttempts": 5, // 0 = ilimitados
    "lockMinutes": 15 // 0 = bloqueo indefinido
  }
}
```

Fuente: CONSUMIR_TENANT_CONFIG.md §1

auth-ms es la autoridad sobre los límites de usuarios

El microservicio aplica `maxUsers` y `limits.roles.<CODE>` de forma central y **fail-closed** al crear usuarios (individual, carga masiva y `createExternalUser`). Si el tenant excede el tope, responde **409 Conflict** con el límite en el mensaje. Tu aplicación puede pre-validar con `getTenantConfig` para buena experiencia de usuario, pero debe manejar el 409 y no debe crear usuarios por fuera de auth-ms.

7.2 Patrón de consumo recomendado

- 1 Consultar y cachear**

Mantén la config en memoria con TTL de 5–15 minutos por `tenantId`. No consultes en cada operación de negocio.
- 2 Invalidar por versión**

Usa `version` / `updatedAt` para invalidar la caché cuando el SuperAdmin cambie una política.
- 3 Resistir la caída del servicio**

Si auth-ms no responde, aplica **fail-open** (permitir la operación) y deja un log indicando que se usó caché o defaults. Así una caída del servicio de autenticación no detiene tu operación de negocio.
- 4 Reportar el consumo**

Envía deltas por lote con `reportId` (UUID) para idempotencia. Calcula el corte mensual en la **zona horaria del tenant**, no en UTC.

Convención de claves

`tenantId` es el `id` de la Company (ej. 00000000), no su `_id` de Mongo. `company` es el nombre (ej. BPONET).

08 Alta de una aplicación nueva

Checklist completo para conectar una aplicación desde cero

#	Paso	Responsable	Detalle
1	Registrar los orígenes SSO	Equipo auth-ms	Añadir tu dominio a <code>SSO_ALLOWED_ORIGINS</code> en el <code>.env</code> de auth-ms (separado por comas) y reiniciar el servicio. Sin esto, la redirección no funcionará.
2	Acordar la <code>SERVICE_API_KEY</code>	Equipo auth-ms + seguridad	Generar un secreto fuerte y compartirlo por un canal seguro. Debe ser idéntico en auth-ms y en tu aplicación. No la subas a un repositorio.
3	Crear el módulo en auth-ms	SuperAdmin	Crear el módulo con las rutas de tu aplicación y asignarlo a los roles que deban acceder. Anota el <code>name</code> exacto (ej. <code>crmEducative</code>): será tu <code>storageKey</code> .
4	Configurar las variables del cliente	Tu equipo	<code>USER_MS_HOST</code> , <code>USER_MS_PORT</code> , <code>SERVICE_API_KEY</code> y, si aplica, <code>TLS_*</code> .
5	Configurar el frontend	Tu equipo	Definir <code>authUrl</code> , <code>redirectUri</code> , <code>url</code> y <code>storageKey</code> en tu <code>environment</code> . Ver sección 3.1.
6	Implementar el flujo del frontend	Tu equipo	Redirección, guard, <code>processAuthData</code> , validación de módulo e interceptor de refresh. Ver sección 3.2.
7	Registrar el cliente TCP (si aplica)	Tu equipo	Añadir <code>TcpClientModule</code> al <code>AppModule</code> . Ver sección 5.1.
8	Validar la política de CORS	Equipo auth-ms	Si tu frontend llama por REST, asegurar que tu origen esté en <code>CORS_ORIGIN</code> de auth-ms.
9	Probar extremo a extremo	Tu equipo	Ejecutar los comandos de la sección 9.1: login, perfil, refresh, política del tenant y un comando TCP.
10	Asignar usuarios de prueba	SuperAdmin	Crear usuarios en la empresa correcta y asignarles un rol que incluya el módulo. Verifica también los límites del tenant (<code>maxUsers</code> , cuotas) para que no bloqueen las pruebas.

Los dos errores más comunes al integrar por primera vez

1. El dominio no está en `SSO_ALLOWED_ORIGINS` → el usuario se autentica pero nunca llega a tu app.
2. El `name` del módulo en auth-ms no coincide con `storageKey` → el usuario entra pero ve "No tienes permisos para acceder a esta módulo".

09 Pruebas y troubleshooting

Cómo verificar la integración y qué hacer cuando falla

9.1 Comandos de verificación

```
# 1) Login por REST → debe devolver accessToken y refreshToken
curl -X POST "$AUTH_URL/auth/login" \
  -H "Content-Type: application/json" \
  -d '{"email":"usuario@empresa.com","password":"clave"}'

# 2) Perfil con el token → debe devolver modules, roles y permissions
curl "$AUTH_URL/users/profile" -H "Authorization: Bearer $ACCESS_TOKEN"

# 3) Refresco → devuelve un accessToken nuevo
curl -X POST "$AUTH_URL/auth/refresh" \
  -H "Content-Type: application/json" \
  -d '{"refreshToken":"$REFRESH_TOKEN"}'

# 4) Perfil como servicio (clave compartida)
curl "$AUTH_URL/users/profile?userId=$USER_ID" -H "x-service-key: $SERVICE_API_KEY"

# 5) Política del tenant
curl "$AUTH_URL/tenants/config?tenantId=0000000" -H "x-service-key: $SERVICE_API_KEY"

# 6) TCP → ejecutar el script de la sección 5.6
node test-tcp.js
```

9.2 Tabla de síntomas y soluciones

Síntoma	Causa probable	Solución
431 Request Header Fields Too Large / 414 URI Too Long	Token con el árbol de permisos incrustado (legado) o URL de redirección demasiado larga.	Usar el token ligero (<code>buildIdentityPayload</code>) y mover la autorización a <code>GET /users/profile</code> .
401 Invalid service credentials	<code>SERVICE_API_KEY</code> distinta, ausente en el payload, o con espacios sobrantes.	Verificar que coincida con el <code>.env</code> de <code>auth-ms</code> y que viaje dentro del payload TCP (no en un header).
401 en las peticiones del frontend	Access token expirado y el interceptor no encuentra el refresh token.	Confirmar que <code>refreshToken</code> está en <code>sessionStorage</code> . Si el usuario cerró la pestaña, deberá iniciar sesión de nuevo (comportamiento esperado).
El usuario se autentica pero nunca llega a mi app	Su dominio no está en <code>SSO_ALLOWED_ORIGINS</code> .	Registrar el dominio y reiniciar <code>auth-ms</code> . Verificar que <code>redirectUri</code> use el mismo dominio exacto.
"No tienes permisos para acceder a este módulo"	<code>storageKey</code> no coincide con el <code>name</code> del módulo, o el usuario no tiene el módulo asignado.	Comparar ambos valores (distingue mayúsculas). Asignar el módulo al rol del usuario desde la consola de administración.
CORS bloquea la petición desde el navegador	El origen no está en <code>CORS_ORIGIN</code> , o se usa <code>x-service-key</code> (no está en <code>allowedHeaders</code>).	Agregar el origen a <code>CORS_ORIGIN</code> . Para <code>x-service-key</code> desde el navegador, añadirlo a <code>allowedHeaders</code> en <code>main.ts</code> .
ECONNREFUSED al puerto 3011	<code>auth-ms</code> no escucha TCP, o <code>host</code> /puerto incorrectos.	Verificar <code>MICROSERVICE_HOST/MICROSERVICE_PORT</code> en <code>auth-ms</code> y el firewall. Probar con el script de la sección 5.6.
Login falla con <code>TRIAL_EXPIRED</code>	El período de prueba del usuario venció (<code>limits.trialDays</code> o <code>TEST_USER_DAYS</code>).	Ampliar la política del tenant o desactivar la marca de prueba del usuario. <code>0</code> = sin expiración.
Login falla con <code>COMPANY_BLOCKED</code>	La empresa está bloqueada (temporal o indefinidamente).	Levantar el bloqueo desde la consola: <code>PATCH /companies/:id/unblock</code> .
429 Too Many Requests	Throttling global: 10 req/min por defecto; el login es más estricto (5/min).	Aplicar <i>backoff</i> . Mostrar un mensaje al usuario en vez de reintentar en bucle.

9.3 Verificar la configuración de auth-ms

Un arranque exitoso debe imprimir en el log las URLs de ambos servidores:

```
Servidor REST: http://[::]:3010  
Swagger docs: http://[::]:3010/api-docs
```

Y la documentación interactiva completa está en [/api-docs](http://[::]:3010/api-docs).

10 Referencias y documentación viva

Dónde consultar más detalle o regenerar la referencia

Documentos del repositorio

Archivo	Contenido
CONSUMIR_TENANT_CONFIG.md	Contrato completo de políticas por tenant y patrón de consumo por aplicación.
ESTRATEGIA_AUTH.md	El problema del token sobredimensionado (431/414) y la solución adoptada.
ESTRATEGIA_SEGURIDAD_TCP.md	Estrategia de seguridad del canal TCP: TLS, mTLS, segmentación de red.
ROADMAP_AUTH_INDUSTRIA.md	Brechas frente a estándares: MFA, OIDC/OAuth2, JWKS, SCIM, auditoría.
PLAN_CONFIG_TENANTS_SUPERADMIN.md	Diseño de la configuración por empresa administrada desde el SuperAdmin.

Documentación generada por el propio servicio

El servicio publica su propia referencia de comandos TCP

Además de Swagger en `/api-docs`, cada dominio expone un endpoint **de solo lectura** que documenta los patrones TCP soportados. No envía datos reales: la comunicación real ocurre por el socket TCP.

```
POST /api/users/tcp-docs/message-patterns
POST /api/roles/tcp-docs/message-patterns
POST /api/permissions/tcp-docs/message-patterns
POST /api/modules/tcp-docs/message-patterns
POST /api/companies/tcp-docs/message-patterns
```

Origen: `auth-ms/src/users/users.controller.ts:793-806`

Código de referencia en el ecosistema

Aplicación	Ruta	Qué aporta
crm-educative	auth-guard.ts process-auth-data.ts auth.interceptor.ts	Flujo SSO completo desde el navegador
educative-backend	core/tcp/tcp.module.ts core/services/auth.service.ts	Cliente TCP simple y llamadas con serviceKey
api-whatsapp	core/tcp/tcp.module.ts core/strategies/jwt.strategy.ts	Cliente TCP con TLS y validación de JWT en cada request
auth-ms	src/main.ts src/core/guards/*	Arranque híbrido, guards de servicio y TCP

Resumen en una línea

Registra tu dominio en `SSO_ALLOWED_ORIGINS` , comparte la `SERVICE_API_KEY` , apunta `storageKey` al módulo correcto, y usa la redirección SSO para usuarios y el canal TCP con `serviceKey` para tu backend.